

✓ D-1 PERCOBAAN HISTOGRAM

```
import cv2 as cv
from google.colab.patches import cv2_imshow
from skimage import io
import matplotlib.pyplot as plt
import numpy as np
import math
import os
import glob
```

```
# membuat histogram image (manual)
img = cv.imread('/content/drive/MyDrive/PCVK/lena_2.jpg')
img = cv.cvtColor(img, cv.COLOR_BGR2RGB)

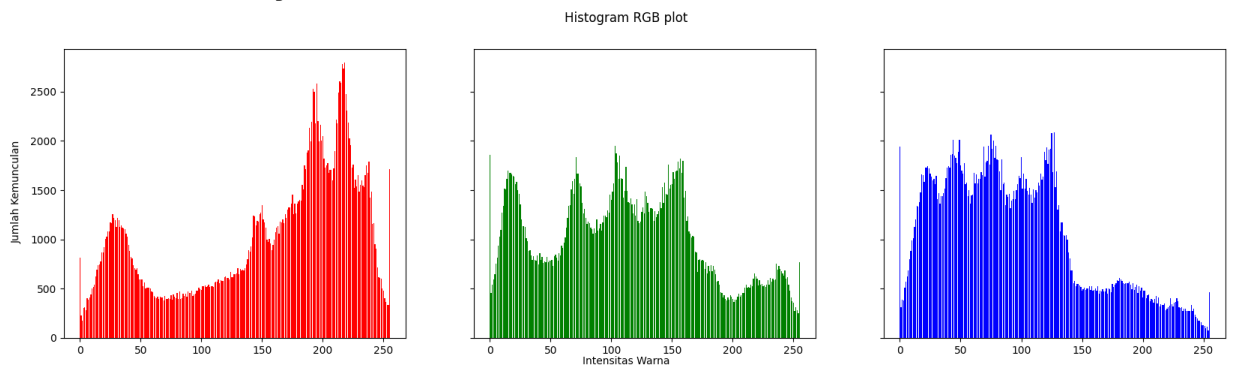
height, width, depth = np.shape(img)
names = np.arange(256)

red   = [0]*256
green = [0]*256
blue  = [0]*256

for y in range(0, height):
    for x in range(0, width):
        red[img[y][x][0]] += 1
        green[img[y][x][1]] += 1
        blue[img[y][x][2]] += 1

names = np.arange(256)
fig, axs = plt.subplots(1, 3, figsize=[20,5], sharex=True, sharey=True)
fig.suptitle('Histogram RGB plot')
fig.text(0.09, 0.5, 'Jumlah Kemunculan', va='center', rotation='vertical')
fig.text(0.5, 0.04, 'Intensitas Warna', ha='center')
axs[0].bar(names, red, color='red')
axs[1].bar(names, green, color='green')
axs[2].bar(names, blue, color='blue')
```

<BarContainer object of 256 artists>



✓ PERTANYAAN PRAKTIKUM D1

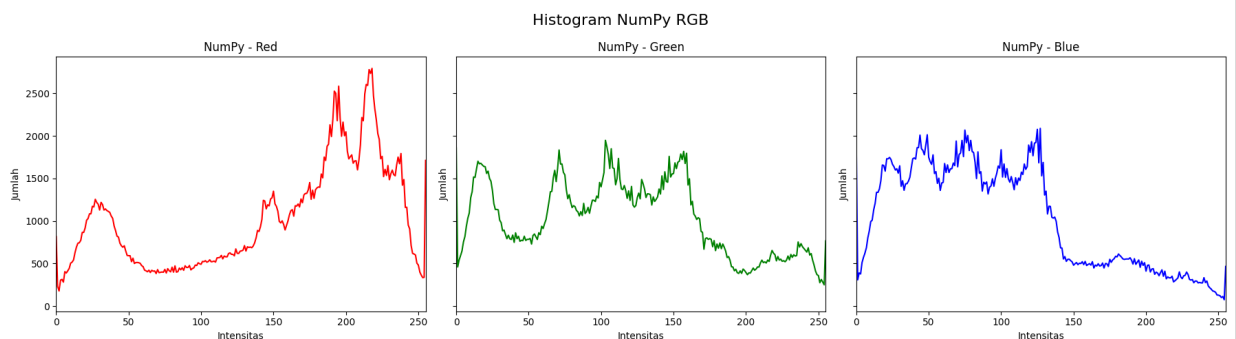
- Buatlah histogram citra yang sama akan tetapi menggunakan library yang dimiliki oleh NumPy yaitu "histogram". Bandingkan hasilnya. Apakah output muncul sama?

```
# Histogram dengan NumPy
hist_r, _ = np.histogram(img[:, :, 0].ravel(), bins=256, range=(0, 256))
hist_g, _ = np.histogram(img[:, :, 1].ravel(), bins=256, range=(0, 256))
hist_b, _ = np.histogram(img[:, :, 2].ravel(), bins=256, range=(0, 256))

# Plot hasil NumPy
fig, axs = plt.subplots(1, 3, figsize=(18, 5), sharex=True, sharey=True)
axs[0].plot(hist_r, color='red'); axs[0].set_title("NumPy - Red")
axs[1].plot(hist_g, color='green'); axs[1].set_title("NumPy - Green")
axs[2].plot(hist_b, color='blue'); axs[2].set_title("NumPy - Blue")

for ax in axs.flat:
    ax.set_xlim([0, 255])
    ax.set_xlabel("Intensitas")
    ax.set_ylabel("Jumlah")

plt.suptitle("Histogram NumPy RGB", fontsize=16)
plt.tight_layout()
plt.show()
```



Histogram citra yang dibuat menggunakan perhitungan manual dan menggunakan fungsi `numpy.histogram()` menghasilkan output yang sama, karena keduanya sama-sama menghitung jumlah kemunculan piksel pada setiap tingkat intensitas (0–255). Perbedaan hanya terletak pada cara penyajian grafiknya. Pada metode manual, histogram ditampilkan dengan batang berwarna sesuai kanal (R, G, B), sedangkan dengan NumPy umumnya ditampilkan dalam bentuk garis (default hitam atau sesuai warna yang dipilih).

Dengan demikian, dapat disimpulkan bahwa hasil perhitungan histogram

Dengan demikian, dapat disimpulkan bahwa hasil perhitungan histogram manual dan NumPy identik, sehingga penggunaan `numpy.histogram()` dapat mempercepat proses pembuatan histogram tanpa mengubah hasil perhitungannya.

- ▼ Buatlah histogram dengan menggunakan dengan menggunakan image KTM lama.jpg. Analisis distribusi intensitas dari histogram yang dihasilkan apakah gambar cenderung gelap, terang, atau kontras rendah

```
# Daftar file gambar
files = [
    ("ktm_luar_ruangan.jpg", "KTM Luar Ruangan"),
    ("ktm_ruangan_flash.jpg", "KTM Ruangan Flash"),
    ("ktm_ruangan_tanpa_flash.jpg", "KTM Ruangan Tanpa Flash"),
    ("ktm_dalam_gelap_flash.jpeg", "KTM Dalam Gelap + Flash"),
    ("ktm_dalam_gelap.jpeg", "KTM Dalam Gelap")
]

plt.figure(figsize=(15, 10))

for i, (fname, title) in enumerate(files, start=1):
    # Baca gambar grayscale
    img = cv.imread(f"/content/drive/MyDrive/PCVK/{fname}", cv.IMREAD_GRAYSCALE)

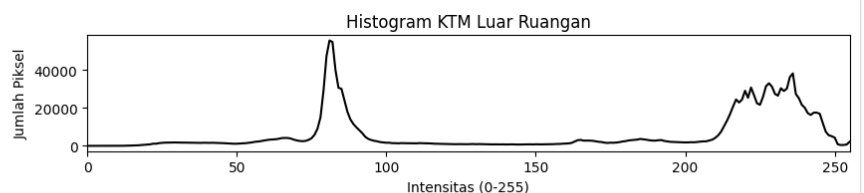
    # Hitung histogram
    hist, bins = np.histogram(img.ravel(), bins=256, range=(0,255))

    # Tampilkan citra
    plt.subplot(len(files), 2, 2*i-1)
    plt.imshow(img, cmap='gray', vmin=0, vmax=255)
    plt.title(f"Citra {title}")
    plt.axis("off")

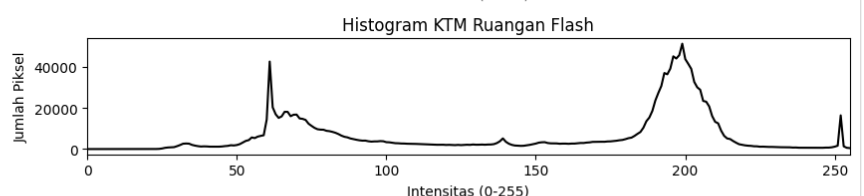
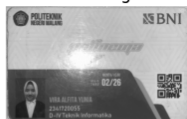
    # Tampilkan histogram
    plt.subplot(len(files), 2, 2*i)
    plt.plot(hist, color='black')
    plt.title(f"Histogram {title}")
    plt.xlabel("Intensitas (0-255)")
    plt.ylabel("Jumlah Pixel")
    plt.xlim([0,255])

plt.tight_layout()
plt.show()
```

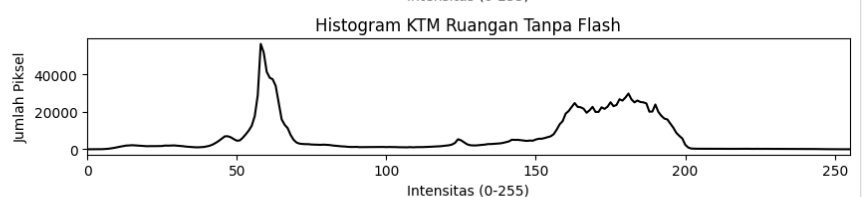
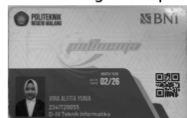
Citra KTM Luar Ruangan



Citra KTM Ruangan Flash



Citra KTM Ruangan Tanpa Flash

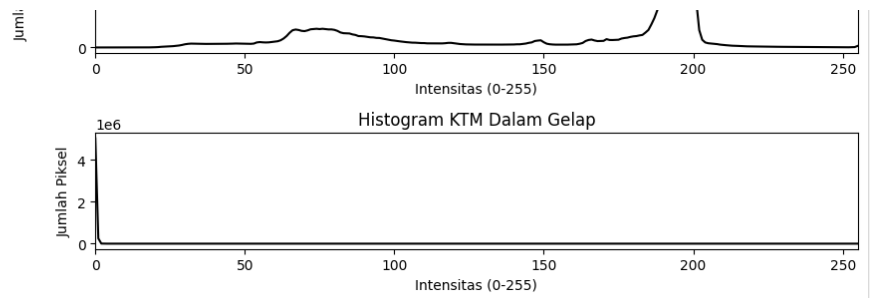


Citra KTM Dalam Gelap + Flash





Citra KTM Dalam Gelap



1. KTM Luar Ruangan

Histogram menyebar luas dari intensitas rendah sampai tinggi.

2. KTM Ruangan dengan Flash

Ada puncak sangat tinggi di intensitas terang (mendekati 255). Ini menunjukkan efek flash sehingga bagian tertentu jadi sangat terang/overexposed. Tapi detail tetap terlihat karena distribusi juga ada di area menengah.

3. KTM Ruangan Tanpa Flash

Histogram lebih condong ke intensitas menengah ke gelap. Puncak di intensitas sekitar 50–100 sehingga pencahayaan lebih redup dibanding pakai flash. Gambar masih cukup terlihat tapi agak gelap.

4. KTM Dalam Gelap + Flash

Histogram dominan di area intensitas terang (sekitar 180–220). Gambar jadi jelas karena flash mendominasi, meski pencahayaan ruangan gelap. Namun distribusi tidak seluas di luar ruangan sehingga menjadi lebih kontras.

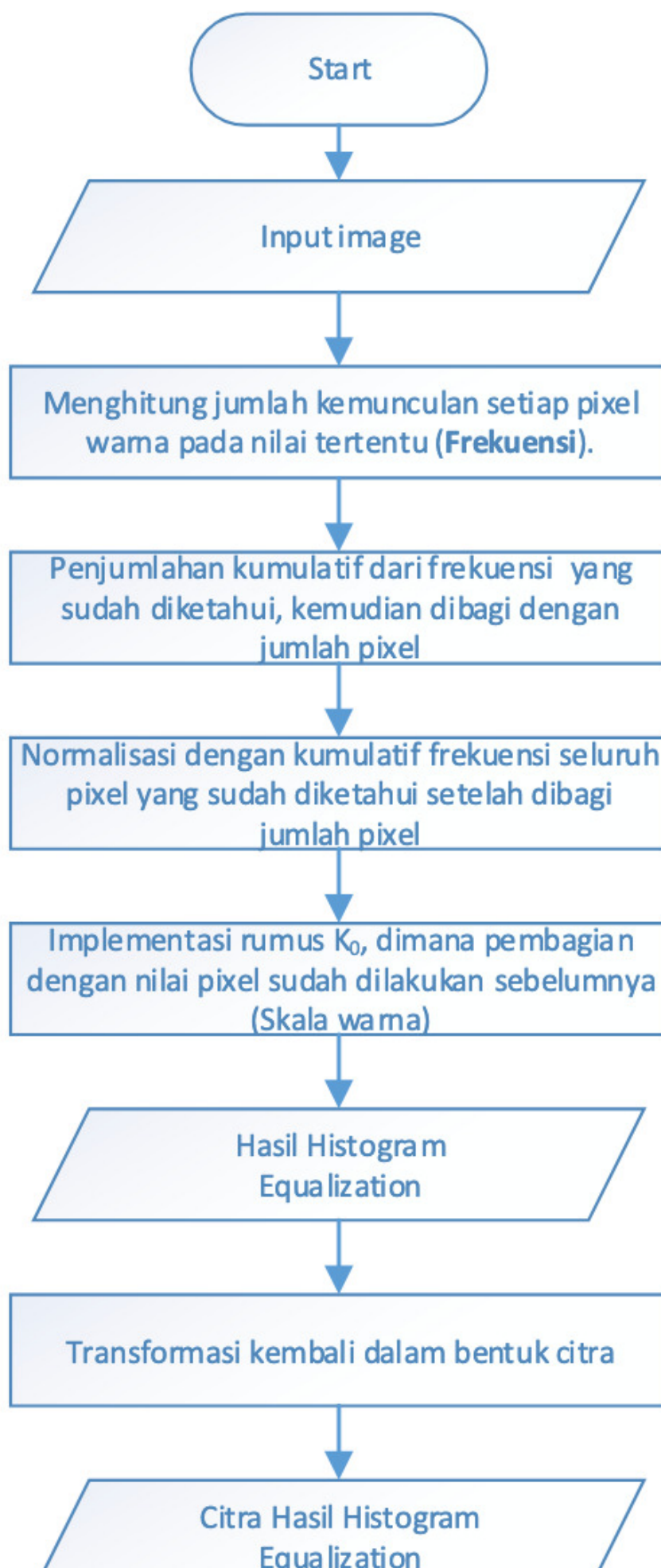
5. KTM Dalam Gelap (tanpa flash)

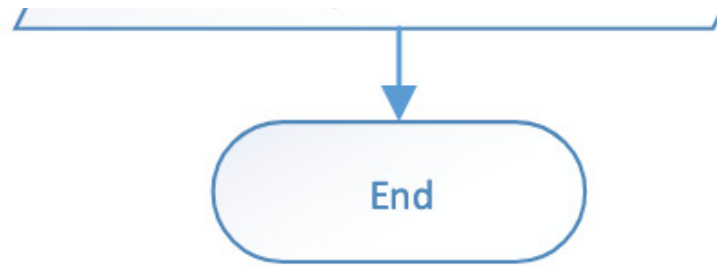
Histogram menumpuk hampir seluruhnya di intensitas sangat rendah

histogram menumpuk hampir seluruhnya di intensitas sangat rendah (sekitar 0–20). Artinya hampir semua piksel gelap → citra hampir hitam total. Informasi visual hilang (tidak terbaca).

✓ D-2 PERCOBAAN HISTOGRAM EQUALIZATION

✓ 1. Hitung sesuai flowchart





```

# --- Input image (RGB utk display) ---
img_bgr = cv.imread('/content/drive/MyDrive/PCVK/lena_lc.jpg')
img = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)

# == Helper: equalization manual untuk 1 channel (0..255) ==
def he_manual_channel(ch):
    # 1) Frekuensi
    hist = np.bincount(ch.ravel(), minlength=256).astype(np.uint32)
    # 2) CDF (kumulatif)
    cdf = hist.cumsum()
    # 3) Normalisasi + 4) K0 (skala 0..255) -> LUT
    cdf_min = cdf[cdf > 0].min() # H_min
    L = 256
    lut = np.round((cdf - cdf_min) * (L - 1) / (cdf[-1] - cdf_min))
    lut = np.clip(lut, 0, 255).astype(np.uint8)
    # 5) Transformasi balik ke citra (apply LUT)
    return lut[ch], hist, np.bincount(lut[ch].ravel(), minlength=256)

# Split RGB
r, g, b = cv.split(img)

# Equalize manual per channel
r_eq, hist_r, hist_r_eq = he_manual_channel(r)
g_eq, hist_g, hist_g_eq = he_manual_channel(g)
b_eq, hist_b, hist_b_eq = he_manual_channel(b)

# Gabungkan kembali
img_eq = cv.merge((r_eq, g_eq, b_eq))

# --- Plot histogram Before/After (per flowchart: cek sebaran) ---
fig, axs = plt.subplots(2, 3, figsize=(20,6), sharex=True, sharey=True)
fig.suptitle('Histogram RGB - Manual HE (Flowchart)', y=1.02)

axs[0,0].bar(range(256), hist_r, color='red');   axs[0,0].set_title('Bef
axs[0,1].bar(range(256), hist_g, color='green'); axs[0,1].set_title('Bef
axs[0,2].bar(range(256), hist_b, color='blue');  axs[0,2].set_title('Bef

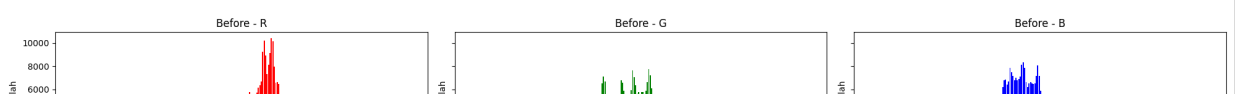
axs[1,0].bar(range(256), hist_r_eq, color='red');   axs[1,0].set_title('
axs[1,1].bar(range(256), hist_g_eq, color='green'); axs[1,1].set_title('
axs[1,2].bar(range(256), hist_b_eq, color='blue');  axs[1,2].set_title('

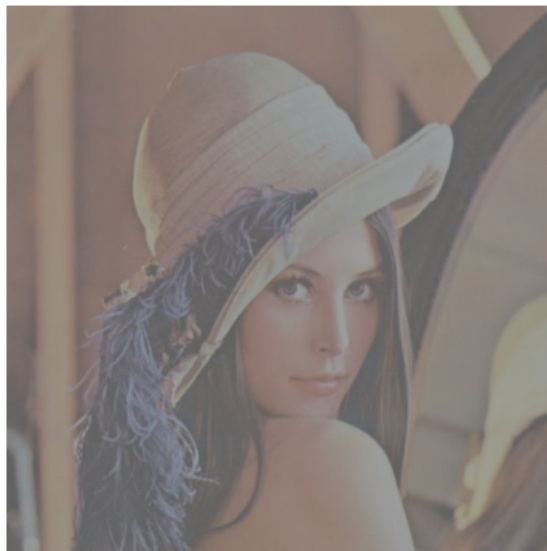
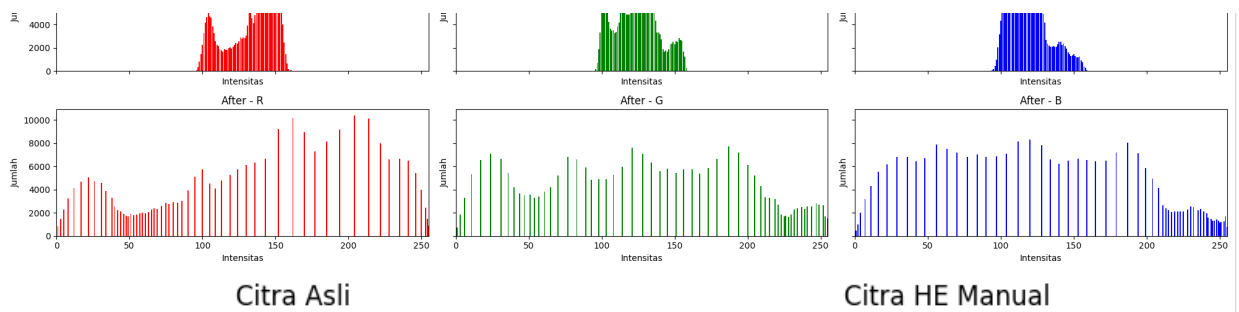
for ax in axs.flat:
    ax.set_xlim([0,255]); ax.set_xlabel('Intensitas'); ax.set_ylabel('Ju
plt.tight_layout(); plt.show()

# --- Tampilkan citra ---
plt.figure(figsize=(10,5))
plt.subplot(1,2,1); plt.imshow(img);   plt.title('Citra Asli');      p
plt.subplot(1,2,2); plt.imshow(img_eq); plt.title('Citra HE Manual'); pl
plt.show()

```

Histogram RGB - Manual HE (Flowchart)





2. Menggunakan equalizeHist

```

# Baca citra berwarna
img = cv.imread('/content/drive/MyDrive/PCVK/lena_lc.jpg')
img = cv.cvtColor(img, cv.COLOR_BGR2RGB)

# Split channel
r, g, b = cv.split(img)

# Equalize tiap channel
r_eq = cv.equalizeHist(r)
g_eq = cv.equalizeHist(g)
b_eq = cv.equalizeHist(b)

# Gabungkan hasil
img_eq = cv.merge((r_eq, g_eq, b_eq))

# Hitung histogram sebelum & sesudah
hist_r, _ = np.histogram(r.ravel(), bins=256, range=(0,256))
hist_g, _ = np.histogram(g.ravel(), bins=256, range=(0,256))
hist_b, _ = np.histogram(b.ravel(), bins=256, range=(0,256))
hist_r_eq, _ = np.histogram(r_eq.ravel(), bins=256, range=(0,256))
hist_g_eq, _ = np.histogram(g_eq.ravel(), bins=256, range=(0,256))
hist_b_eq, _ = np.histogram(b_eq.ravel(), bins=256, range=(0,256))

```

```
nist_p_eq, _= np.histogram(p_eq.ravel(),bins=256, range=(0,256))

# Plot 2 baris x 3 kolom
fig, axs = plt.subplots(2, 3, figsize=(20,6), sharex=True, sharey=True)
fig.suptitle('Histogram RGB Before & After Equalization')

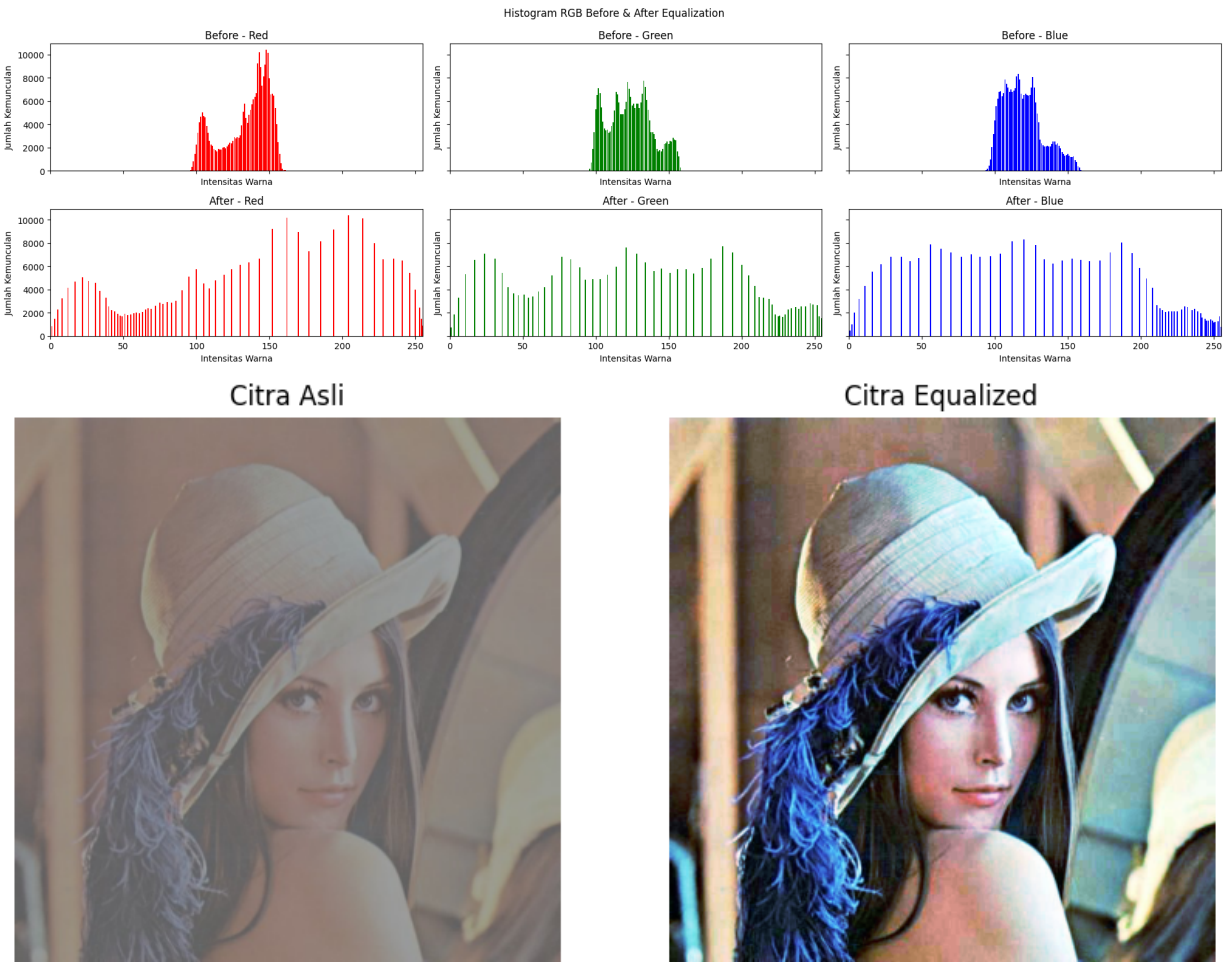
# Baris 1 = Before
axs[0,0].bar(range(256), hist_r, color='red');   axs[0,0].set_title('Bef
axs[0,1].bar(range(256), hist_g, color='green'); axs[0,1].set_title('Bef
axs[0,2].bar(range(256), hist_b, color='blue');   axs[0,2].set_title('Bef

# Baris 2 = After
axs[1,0].bar(range(256), hist_r_eq, color='red');   axs[1,0].set_title('
axs[1,1].bar(range(256), hist_g_eq, color='green'); axs[1,1].set_title('
axs[1,2].bar(range(256), hist_b_eq, color='blue');   axs[1,2].set_title('

for ax in axs.flat:
    ax.set_xlim([0,255])
    ax.set_xlabel("Intensitas Warna")
    ax.set_ylabel("Jumlah Kemunculan")

plt.tight_layout()
plt.show()

# Tampilkan citra sebelum & sesudah equalization
plt.figure(figsize=(10,5))
plt.subplot(1,2,1); plt.imshow(img);   plt.title("Citra Asli");   p
plt.subplot(1,2,2); plt.imshow(img_eq); plt.title("Citra Equalized"); pl
plt.show()
```



▼ PERTANYAAN PRAKTIKUM D2

▼ Lena.jpg - Equalization & PSNR

```
# --- A. Histogram Equalization (grayscale) ---
lena = cv.imread('/content/drive/MyDrive/PCVK/lena_2.jpg', cv.IMREAD_GRAYSCALE)
assert lena is not None, "Cek path ke lena_2.jpg"

lena_eq = cv.equalizeHist(lena)    # <- pakai OpenCV

# PSNR (asli vs equalized)
def psnr(x, y, maxval=255.0):
    x = x.astype(np.float64); y = y.astype(np.float64)
    mse = np.mean((x - y)**2)
    return float('inf') if mse == 0 else 10*np.log10((maxval**2)/mse)

psnr_val = psnr(lena, lena_eq)
print(f"PSNR (Lena vs Lena_EQ): {psnr_val:.2f} dB")

# Menampilkan citra
plt.figure(figsize=(10,5))
plt.subplot(1,2,1); plt.imshow(lena, cmap='gray');    plt.title('Lena Asli')
plt.subplot(1,2,2); plt.imshow(lena_eq, cmap='gray'); plt.title('Lena Equalized')
plt.show()
```

PSNR (Lena vs Lena_EQ): 24.99 dB

Lena Asli



Lena Equalized

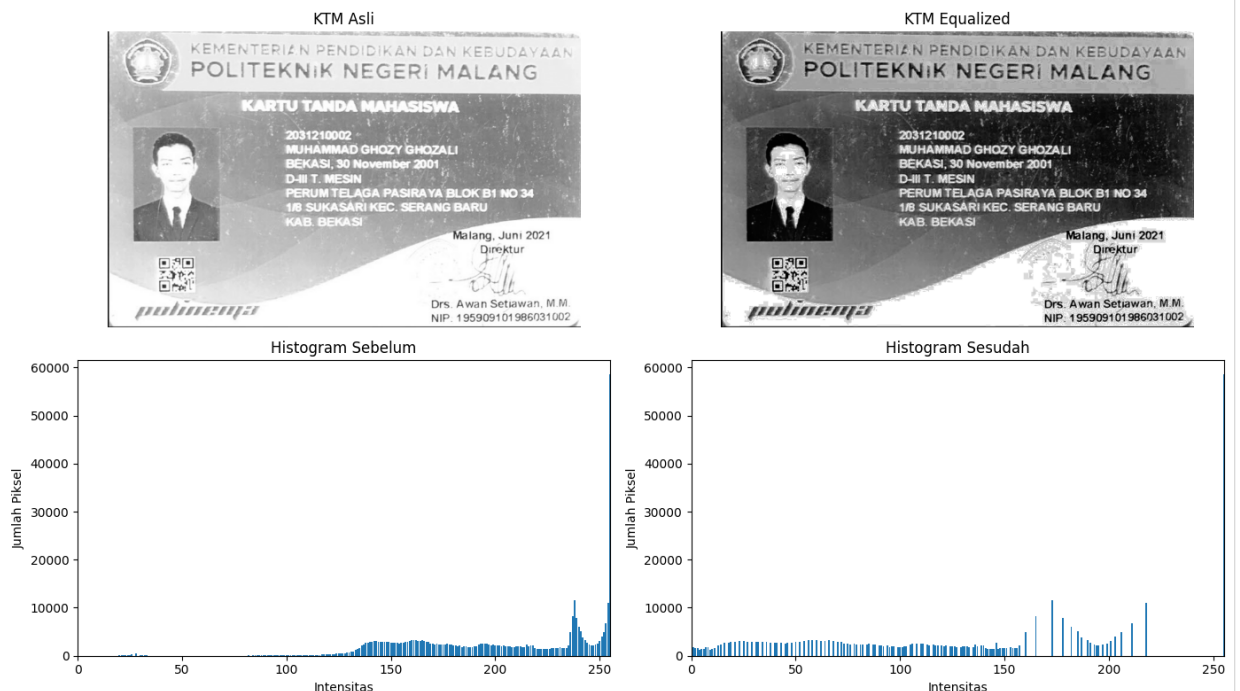


▼ KTM lama.jpg - Equalization + Layout histogram

```
# Equalize (grayscale)
ktm = cv.imread('/content/drive/MyDrive/PCVK/KTM lama.jpg', cv.IMREAD_GRAYSCALE)
assert ktm is not None, "Cek path ke KTM lama.jpg"
ktm_eq = cv.equalizeHist(ktm)
```

```
# Histogram sebelum & sesudah
hist_before, _ = np.histogram(ktm.ravel(), bins=256, range=(0,256))
hist_after, _ = np.histogram(ktm_eq.ravel(), bins=256, range=(0,256))

# Layout: 2 gambar + 2 histogram (1 canvas)
fig = plt.figure(figsize=(14,8))
ax1 = plt.subplot2grid((2,2),(0,0)); ax1.imshow(ktm, cmap='gray'); ax
ax2 = plt.subplot2grid((2,2),(0,1)); ax2.imshow(ktm_eq, cmap='gray'); ax
ax3 = plt.subplot2grid((2,2),(1,0)); ax3.bar(range(256), hist_before); a
ax4 = plt.subplot2grid((2,2),(1,1)); ax4.bar(range(256), hist_after); a
for ax in (ax3, ax4):
    ax.set_xlabel('Intensitas'); ax.set_ylabel('Jumlah Piksel')
plt.tight_layout(); plt.show()
```



Dengan menerapkan proses equalize, gambar yang dihasilkan lebih terang dan jelas sehingga informasi identitas mahasiswa dapat ditampilkan dengan grayscale yang lebih kontras

✓ D-3 TUGAS PRAKTIKUM DITHERING

Lakukanlah proses dithering Floyd and Steinberg seperti output berikut (tampilan image awal, dan tampilan setelah

dithering) berdasarkan flowchart di bagian bawah halaman modul ini! (Gunakan gambar lena.jpg)

```
src = cv.imread('/content/drive/MyDrive/PCVK/lena.jpg', cv.IMREAD_GRAYSC

# Floyd–Steinberg dithering
img = src.astype(np.float32).copy()
h, w = img.shape

for y in range(h):
    for x in range(w):
        old_pixel = img[y, x]
        new_pixel = 255 if old_pixel >= 128 else 0
        img[y, x] = new_pixel
        error = old_pixel - new_pixel

        # Sebarkan error ke tetangga (sesuai bobot FS)
        if x + 1 < w:
            img[y, x + 1] += error * (7 / 16)
        if y + 1 < h and x - 1 >= 0:
            img[y + 1, x - 1] += error * (3 / 16)
        if y + 1 < h:
            img[y + 1, x] += error * (5 / 16)
        if y + 1 < h and x + 1 < w:
            img[y + 1, x + 1] += error * (1 / 16)

        # Pastikan nilai tetap dalam range 0–255
        img = np.clip(img, 0, 255)

dithered = img.astype(np.uint8)

# Tampilkan hasil
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(src, cmap="gray")
plt.title("Citra Asli (Grayscale)")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(dithered, cmap="gray")
plt.title("Setelah Dithering Floyd–Steinberg")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Citra Asli (Grayscale)



Setelah Dithering Floyd–Steinberg



Menggunakan lena_lc.jpg rubah menjadi grey image, kemudian terapkan histogram equalization sehingga terjadi perbaikan sebaran warna pada citra. Setelah itu dari hasil histogram equalization implementasikan proses dithering Floyd and Steinberg sehingga menampilkan gambar di bawah ini

```
# path file
img_path = "/content/drive/MyDrive/PCVK/lena_lc.jpg"
bgr = cv.imread(img_path, cv.IMREAD_COLOR); assert bgr is not None
gray = cv.cvtColor(bgr, cv.COLOR_BGR2GRAY)

# histogram equalization
eq = cv.equalizeHist(gray)

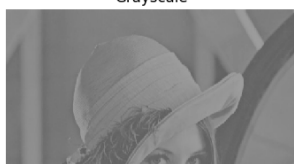
# Floyd–Steinberg (0/255), clamp per tetangga
def floyd_steinberg(u8):
    img = u8.astype(np.float32)
    h, w = img.shape
    for y in range(h):
        for x in range(w):
            old = img[y, x]
            new = 255.0 if old >= 128 else 0.0
            img[y, x] = new
            err = old - new
            if x+1 < w:
                v = img[y, x+1] + err*(7/16); img[y, x+1] = 0.0 if v<0
            if y+1 < h and x-1 >= 0:
                v = img[y+1, x-1] + err*(3/16); img[y+1, x-1] = 0.0 if v
            if y+1 < h:
                v = img[y+1, x] + err*(5/16); img[y+1, x] = 0.0 if v
            if y+1 < h and x+1 < w:
                v = img[y+1, x+1] + err*(1/16); img[y+1, x+1] = 0.0 if v
    return img.astype(np.uint8)

dither = floyd_steinberg(eq)

# tampilkan (nearest biar pola titik jelas)
scale = 1.8
big = cv.resize(dither, None, fx=scale, fy=scale, interpolation=cv.INTER

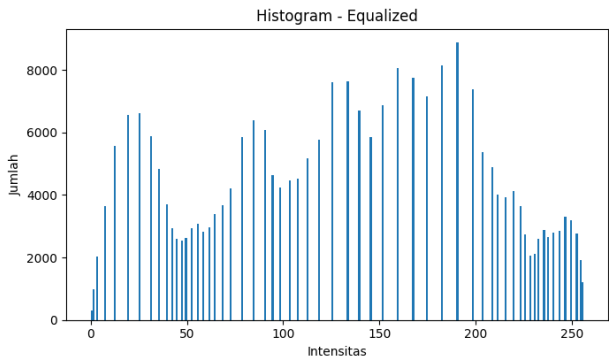
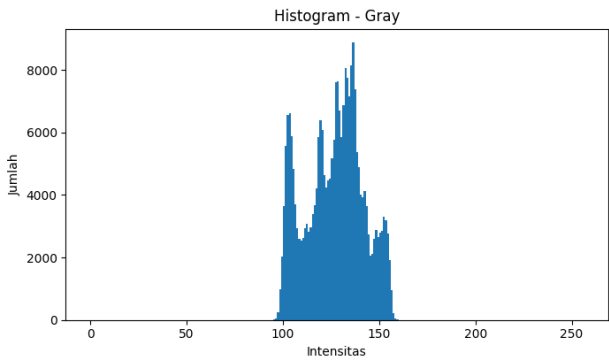
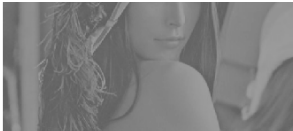
plt.figure(figsize=(14,12))
plt.subplot(3,2,1); plt.imshow(gray, cmap='gray', vmin=0, vmax=255, inte
plt.subplot(3,2,2); plt.imshow(eq, cmap='gray', vmin=0, vmax=255, inte
plt.subplot(3,2,3); plt.hist(gray.ravel(), bins=256, range=(0,256)); plt
plt.subplot(3,2,4); plt.hist(eq.ravel(), bins=256, range=(0,256)); plt
plt.subplot(3,1,3); plt.imshow(big, cmap='gray', vmin=0, vmax=255, inter
plt.tight_layout(); plt.show()
```

Grayscale



Equalization





Dithering Floyd-Steinberg



